

AI Football Scouting Assistant -- Project Case Study

Serverless AI Analytics: Python Deterministic Stats Engine + LangChain ReAct Orchestration

GitHub Repository: <https://github.com/Creepling/ai-football-scouting-bot>

Tech Stack: Python 3.11, Google Cloud Functions (Gen 2), LangChain, Wyscout API v3, MongoDB Atlas, Google Cloud Storage, Matplotlib, Seaborn, Telegram Bot API

1. Problem & Product Context

Initial football scouting and candidate screening requires manual data collation across disparate silos (Transfermarkt for biographical details and Wyscout for raw match events), tedious Per-90 and league-relative percentile normalization, manual chart generation, and narrative drafting.

The product fully automates this scouting workflow: given a raw player profile URL, the system resolves entities, queries match databases, computes deterministic league percentiles, renders visual scouting cards, and synthesizes tactical scout evaluations directly in Telegram.

2. My Role & Engineering Ownership

Sole AI & Backend Engineer: End-to-end architecture and full-lifecycle implementation -- from mathematical metric normalization and MongoDB aggregation pipelines to LangChain ReAct agents, Matplotlib rendering engines, and GCP serverless infrastructure.

3. System Architecture & Core Philosophy

Core Philosophy (Deterministic Truth vs. Cognitive Synthesis): Strict separation between mathematical computation and natural language generation. All statistical metrics, Per-90 conversions, percentile ranks (0-100), and Bayesian adjustments are calculated deterministically in Python/Pandas. The LLM is never used as a source of numerical truth (numerical calculations are kept outside the LLM path) and is strictly confined to fuzzy entity resolution, autonomous tool calling (ReAct loop), and qualitative tactical narrative synthesis.

Dataflow Pipeline: Telegram Webhook -> GCP Cloud Function Router -> Thread-Safe Context (contextvars) -> LangChain ReAct Agent -> External APIs (Wyscout API v3, MongoDB Atlas, Serper API) -> Deterministic Engine (Pandas / NumPy) -> In-Memory Viz Engine (Matplotlib / Seaborn) -> GCS Session Store -> Telegram Bot API.

4. What I Personally Built

- Modular Serverless Architecture: Decoupled Python service deployed on Google Cloud Functions (Gen 2 / Cloud Run) with thread-safe session isolation via contextvars.
- Deterministic Stats Engine: Implemented Per-90 normalization formulas, league-position percentile distributions, and Bayesian smoothing (bayes_rank).
- Data Pipelines & Aggregations: Multi-stage MongoDB Atlas aggregation pipelines grouping raw match events across seasons, paired with custom Wyscout API v3 REST wrappers.
- AI Tool Registry: 9 specialized LangChain tools (URL scraping, season switching, metric deep-dives, heatmaps, position breakdown).
- Automated Visual Generator: In-memory Matplotlib & Seaborn rendering pipeline generating scouting cards, peer histograms, and heatmaps with zero memory leaks.
- DevOps & Security: Zero-secret deployment via GCP Secret Manager, CI/CD in GitHub Actions using Workload Identity Federation (WIF), and Telegram webhook secret token verification.

5. Key Technical Challenges & Solutions

- Small-Sample Variance & Outliers: Players with 2 successful dribbles on 2 attempts skewed percentiles to 100%. Solved by engineering a Bayesian weighted formula incorporating league 80th-percentile priors: $(R \cdot v + C \cdot m) / (v + m)$.
- Fuzzy Cross-Platform Resolution: Unstructured Transfermarkt URLs with localized transliterations mapped to Wyscout's rigid search index via a two-stage pipeline (Google Serper API + LLM biographical extraction + Wyscout search indexing).
- Serverless Memory Management: Eliminated container memory bloat in reusable Cloud Function instances by streaming plots directly in io.BytesIO buffers and enforcing deterministic plt.close(fig) deallocations.

6. Results & Production Impact

- End-to-End Automation: Fully automated scouting intelligence pipeline from incoming URL to visual cards and tactical reports without human intervention.
- Multi-Turn Interactive UX: Bi-lingual (EN/RU) inline keyboard menus enabling granular drill-down into specific seasons, isolated metrics, and position distributions.
- Production Stability: Stateless, thread-safe, and secure cloud architecture scaling to zero with no hardcoded credentials.

7. What I Would Do Differently Today

- Decoupled Asynchronous Queue (Cloud Tasks / Pub/Sub): Offload heavy multi-season MongoDB aggregations to background workers with push callbacks to eliminate Telegram webhook timeout risks.
- Vector Similarity Search (pgvector / Qdrant): Embed normalized percentile vectors to enable automated 'Find statistical twins / replacement candidates' search across leagues.
- Redis Caching Layer: Implement distributed caching for static Wyscout profiles and seasonal baselines to minimize third-party API costs.
- Modern LangChain LCEL: Migrate from string-based ReAct agent parsing to Pydantic Function Calling with structured outputs.